

Efektywna reprezentacja słownika jako minimalny automat deterministyczny

Biblioteka Lexicon w C++20
Zaawansowane Biblioteki Programistyczne

Piotr Marcol

26.05.2026

Plan prezentacji

- 1 Wprowadzenie i API
- 2 Reprezentacja danych
- 3 Budowa i minimalizacja
- 4 Operacje na słowniku
- 5 Wyniki i ograniczenia

- reprezentacja zbioru słów jako minimalny automat deterministyczny,
- szybkie sprawdzanie, czy słowo należy do słownika,
- ograniczenie redundancji przez współdzielenie prefiksów i sufiksów,
- API zbliżone do kontenera STL,
- możliwość eksportu struktury do Graphviz DOT.

Struktura projektu

- `include/lexicon/lexicon.hpp` – publiczne API biblioteki,
- `src/lexicon.cpp` – implementacja automatu,
- `examples/main.cpp` – program demonstracyjny,
- `CMakeLists.txt` – konfiguracja budowania projektu.

- API kontenerowe:
 - `contains`, `find`, `count`,
 - `begin/end`, `size`, `empty`, `clear`.
- API diagnostyczne:
 - liczba stanów,
 - liczba przejść,
 - szacowane zużycie pamięci.
- API wizualizacji:
 - `exportToDot()`,
 - `exportToDotFile(path)`.
- API budowy:
 - `buildFromSorted(words)`.

Założenie

Lexicon publicznie zachowuje się jak kontener słów, ale wewnętrznie jest minimalnym automatem.

- użytkownik biblioteki nie operuje bezpośrednio na stanach,
- iterator przechodzi po słowach, nie po węzłach grafu,
- automat pozostaje ukryty za prostym API.

Reprezentacja stanu

```
using StateId = std::size_t;
using Symbol = char32_t;

struct Transition {
    Symbol symbol{};
    StateId target{invalidState};
};

struct State {
    bool isFinal = false;
    std::vector<Transition> transitions;
};
```

- StateId wskazuje stan w wektorze states_
- State nie przechowuje własnego identyfikatora,
- przejście zawiera symbol i identyfikator stanu docelowego.

- `std::string` przechowuje tekst jako bajty,
- polskie znaki w UTF-8 zajmują więcej niż jeden bajt,
- przejście automatu powinno odpowiadać znakowi, a nie bajtowi,
- dlatego symbol automatu ma typ `char32_t`.

Efekt

ł, ą, ż są traktowane jako pojedyncze symbole.

CaseMode i normalizacja

```
enum class CaseMode {  
    Sensitive,  
    Insensitive  
};
```

```
explicit Lexicon(CaseMode mode = CaseMode::Insensitive);
```

- tryb jest ustawiany podczas tworzenia słownika,
- normalizacja działa przy budowie i przy wyszukiwaniu,
- Insensitive: ASCII + polskie małe litery,
- pełne Unicode case folding nie jest implementowane.

Założenie: wejście posortowane

- metoda `buildFromSorted()` wymaga słów w porządku leksykograficznym,
- porządek dotyczy postaci po normalizacji,
- kolejne słowo porównywane jest z poprzednim,
- wspólny prefiks określa, którą część poprzedniej ścieżki można minimalizować.

- 1 normalizacja słowa,
- 2 sprawdzenie kolejności i pominięcie duplikatu,
- 3 znalezienie wspólnego prefiksu z poprzednim słowem,
- 4 minimalizacja sufiksu poprzedniej ścieżki,
- 5 dodanie brakujących stanów dla aktualnego słowa,
- 6 oznaczenie ostatniego stanu jako finalnego.

Budowa automatu

```
void Lexicon::buildFromSorted(  
    const std::vector<std::string>& words  
) {  
    clear();  
  
    root_ = createState(false);  
    for (const auto& word : words) {  
        insertSorted(normalize(word));  
    }  
  
    finish();  
}
```

Finish() wykonuje końcową minimalizację i kompaktację:

- minimalizuje ostatnią niezamkniętą ścieżkę,
- zapisuje metryki przed końcowej minimalizacji,
- wykonuje kompaktację stanów osiągalnych,
- czyści struktury pomocnicze buildera.

Właściwe dodawanie słowa

```
void Lexicon::insertSorted(const SymbolString& word) {  
    if (!previousWord_.empty() && word < previousWord_) {  
        throw std::invalid_argument("Words must be sorted.");  
    }  
  
    if (word == previousWord_) {  
        return;  
    }  
  
    const std::size_t commonPrefix =  
        commonPrefixLength(previousWord_, word);  
  
    minimizeFrom(commonPrefix);  
  
    // dodanie brakującego sufiksu aktualnego słowa  
}
```

- dwa stany mogą zostać scalone, jeśli mają identyczne zachowanie,
- porównywana jest finalność oraz lista przejść,
- przejścia muszą prowadzić do tych samych stanów kanonicznych,
- minimalizacja idzie od końca słów w stronę korzenia.

Efekt

Identyczne sufiksy i podgrafy są przechowywane tylko raz.
Minimalizacja nie zmienia języka automatu, tylko reprezentację.

Sygnatura i rejestr stanów

```
struct StateSignature {  
    bool isFinal = false;  
    std::vector<Transition> transitions;  
};
```

```
std::unordered_map<  
    StateSignature,  
    StateId,  
    SignatureHash  
> registry_;
```

- sygnatura opisuje zachowanie stanu,
- rejestr mapuje sygnaturę na stan kanoniczny,
- powtórzona sygnatura oznacza możliwość scalenia.

replaceOrRegister

```
Lexicon::StateId Lexicon::replaceOrRegister(StateId stateId) {
    sortTransitions(states_[stateId]);

    StateSignature signature;
    signature.isFinal = states_[stateId].isFinal;
    signature.transitions = states_[stateId].transitions;

    const auto it = registry_.find(signature);
    if (it != registry_.end()) {
        return it->second;
    }

    registry_.emplace(std::move(signature), stateId);
    return stateId;
}
```

- po scalaniu część stanów może stać się nieosiągalna,
- iteracyjny DFS od `root_` znajduje stany osiągalne,
- tworzony jest nowy wektor stanów,
- przejścia są przepinane przez mapowanie `oldToNew`,
- po kompaktacji końcowy automat nie zawiera martwych stanów.

```
bool Lexicon::contains(const value_type& word) const {
    const SymbolString normalizedWord = normalize(word);
    StateId current = root_;
    for (const Symbol symbol : normalizedWord) {
        const auto& transitions = states_[current].transitions;
        auto it = std::lower_bound(
            transitions.begin(), transitions.end(), symbol,
            [](const Transition& t, Symbol s) {
                return t.symbol < s;
            });
        if (it == transitions.end() || it->symbol != symbol)
            return false;
        current = it->target;
    }
    return states_[current].isFinal;
}
```

Złożoność: formalnie $O(m \log d)$, praktycznie blisko $O(m)$

Iterator po słowach

```
class const_iterator {
public:
    using iterator_category = std::forward_iterator_tag;
    using value_type = std::string;

    reference operator*() const noexcept;
    pointer operator->() const noexcept;
    const_iterator& operator++();
    const_iterator operator++(int);
private:
    std::vector<Frame> stack_;
    std::u32string currentSymbols_;
    std::string currentWord_;
};
```

- iterator przechodzi po słowach zaakceptowanych przez automat,
- nie tworzy wcześniej listy wszystkich słów,
- stan przejścia po grafie jest przechowywany na stosie.

Użycie iteratora

```
lexicon::Lexicon dictionary;  
  
dictionary.buildFromSorted(words);  
  
for (const auto& word : dictionary) {  
    std::cout << word << '\n';  
}
```

- publiczne API nie ujawnia stanów automatu,
- użytkownik iteruje po słowach, tak jak w kontenerze STL,
- automat pozostaje detalem implementacyjnym.

find, count i słowa kanoniczne

```
auto it = dictionary.find("Apple");
```

```
if (it != dictionary.end()) {  
    std::cout << *it << '\n';  
}
```

```
std::cout << dictionary.count("Apple") << '\n';
```

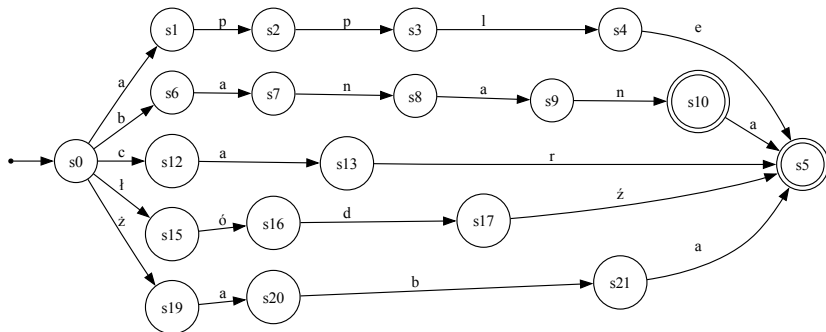
- find() zwraca iterator albo end(),
- count() zwraca 1 albo 0,
- w trybie Insensitive zwracana jest postać kanoniczna.

```
std::string dot = dictionary.exportToDot();
```

```
dictionary.exportToDotFile("graphs/example.dot");
```

- eksportowany jest graf automatu w formacie DOT,
- uwzględniane są tylko stany osiągalne od root_,
- stany finalne oznaczane są jako doublecircle,
- etykiety przejść są zapisywane jako UTF-8.

Przykład eksportu DOT



- `stateCount()` – liczba stanów po finalizacji,
- `reachableStateCount()` – liczba stanów osiągalnych,
- `transitionCount()` – liczba przejść,
- `preMinimizationStateCount()` – stan przed końcową minimalizacją,
- `memoryUsageEstimate()` – szacowane zużycie pamięci.

Uwaga

Pomiar pamięci jest estymacją, a nie dokładnym RSS procesu.

Demo: mały zbiór danych

- kilka przykładowych słów: ASCII + polskie znaki,
- sprawdzenie działania CaseMode,
- sprawdzenie obsługi UTF-8,
- test iteratora i wyszukiwania,
- przykład pomocniczy, nie benchmark wydajności.

Struktura	Elementy / stany	Przejścia	Pamięć
Input	7 słów	–	76 B
<code>std::set</code>	7 słów	–	560 B
<code>std::unordered_set</code>	7 słów	–	544 B
DAFSA przed minimizacją	68 stanów	–	10.88 KiB
DAFSA po minimizacji	63 stany	67	3.17 KiB

Demo na dużym słowniku

Metryka	Wartość
Liczba słów wejściowych	466550
Liczba unikalnych słów	466546
Liczba znaków	4396435
Rozmiar wejścia	4.19 MiB
Stany przed minimalizacją	1356532
Stany po minimalizacji	207535
Przejścia	496498
Szacowana pamięć po minimalizacji	13.91 MiB
Czas budowy	ok. 1.91 s

Etap	Liczba stanów	Szacowana pamięć
Przed minimalizacją	1356532	104.86 MiB
Po minimalizacji	207535	13.91 MiB

Wniosek

Liczba stanów spada ponad sześciokrotnie.

Porównanie z kontenerami standardowymi

- `std::set<std::string>` i `std::unordered_set<std::string>`:
 - przechowują słowa jako osobne napisy,
 - są uniwersalne i łatwe w użyciu.
- `Lexicon`:
 - nie przechowuje każdego słowa jako osobnego stringa,
 - reprezentuje cały zbiór jako automat,
 - współdzieli wspólne fragmenty słów.

Kiedy DAFSA ma sens?

Standardowe kontenery przechowują osobne stringi; `Lexicon` przechowuje wspólną strukturę automatu. Ma to sens dla dużych, statycznych słowników.

- wejście musi być posortowane zgodnie z trybem normalizacji,
- brak dynamicznego insert / erase po finalizacji,
- tryb CaseMode wybierany jest przed budową automatu,
- brak pełnego Unicode case folding,
- pomiary pamięci są estymacją,
- iterator zwraca słowa kanoniczne, a nie oryginalną pisownię.

- pełniejszy benchmark na różnych słownikach,
- serializacja gotowego automatu do pliku,
- wyszukiwanie prefiksowe i autouzupełnianie,
- pełne Unicode case folding przez ICU,
- dokładniejsze pomiary pamięci.

- `Lexicon` publicznie zachowuje się jak kontener słów,
- wewnętrznie używa minimalnego automatu acyklicznego,
- obsługuje UTF-8, tryby `Sensitive/Insensitive` i eksport DOT,
- minimalizacja redukuje liczbę stanów na dużym słowniku,
- projekt pokazuje praktyczne zastosowanie DAFSA w C++20.

Efektywna reprezentacja słownika jako minimalny automat deterministyczny

Biblioteka Lexicon w C++20
Zaawansowane Biblioteki Programistyczne

Piotr Marcol

26.05.2026